

Análise de Predição de Diabetes usando Machine Learning

Autor: Samuel Abrão & João Genaro **Data:** 08/09/2025

Introdução

Este relatório apresenta uma análise de predição de diabetes utilizando técnicas de machine learning. O dataset utilizado contém informações sobre pacientes e suas características clínicas, com o objetivo de prever a ocorrência de diabetes (classe 1) ou não-diabetes (classe 0).

Dataset

O dataset utilizado é o *Diabetes Dataset Missing Data*, contendo as seguintes variáveis:

- Pregnancies:** Número de gestações
- Glucose:** Concentração de glicose no plasma
- BloodPressure:** Pressão arterial diastólica
- SkinThickness:** Espessura da dobra cutânea do tríceps
- Insulin:** Insulina sérica de 2 horas
- BMI:** Índice de massa corporal
- FamilyHistoryDiabetes:** Histórico familiar de diabetes
- Age:** Idade
- Outcome:** Variável alvo (0 = não-diabetes, 1 = diabetes)

Metodologia

Pré-processamento dos Dados

O código implementa as seguintes etapas de pré-processamento:

```
# Leitura do dataset
df = pd.read_excel(xlsx_path)
df.columns = [c.strip().lower() for c in df.columns]

# Conversão de zeros para NaN em variáveis específicas
zeros_as_nan = ['glucose', 'bloodpressure', 'skintickness', 'insulin', 'bmi']
for c in zeros_as_nan:
    df.loc[df[c] == 0, c] = np.nan

# Divisão dos dados
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
```

Modelos Utilizados

Foram testados dois algoritmos de classificação:

- Regressão Logística:** Com regularização L2 e balanceamento de classes
- Random Forest:** Com balanceamento de classes e otimização de hiperparâmetros

Resultados

Comparação dos Modelos

Modelo	Acurácia	Precisão	Recall	F1-Score
Regressão Logística	0.693	0.544	0.731	0.624
Random Forest	0.708	0.573	0.642	0.606

Relatório de Classificação Detalhado

O relatório de classificação para o melhor modelo (Regressão Logística):

Classe	Precisão	Recall	F1-Score	Suporte

Não-Diabetes (0) Classe	0.82 Precisão	0.67 Recall	0.74 F1-Score	125 Suporte
Diabetes (1)	0.54	0.73	0.62	67
Média Macro	0.68	0.70	0.68	192
Média Ponderada	0.73	0.69	0.70	192

Importância das Variáveis

Os coeficientes da regressão logística, indicando a importância de cada variável:

Variável	Coefficiente
Glicose	1.076
BMI	0.706
Gestações	0.419
Histórico Familiar	0.356
Idade	0.273
Espessura da Pele	0.071
Insulina	0.020
Pressão Arterial	-0.101

Visualizações Geradas

As seguintes visualizações foram geradas durante a análise:

- **Matriz de Confusão** (`confusion_matrix.png`): Mostra a distribuição das predições vs. valores reais
- **Curva ROC** (`roc_curve.png`): Demonstra a performance do modelo em diferentes thresholds
- **Curva Precision-Recall** (`pr_curve.png`): Avalia a performance considerando o desbalanceamento das classes
- **Mapa de Correlação** (`correlation_heatmap.png`): Mostra as correlações entre as variáveis

Discussão

Performance dos Modelos

O modelo de Regressão Logística apresentou o melhor F1-score (0.624), sendo selecionado como o modelo final. Apesar do Random Forest ter uma acurácia ligeiramente superior (0.708 vs 0.693), a Regressão Logística demonstrou melhor balanceamento entre precisão e recall.

Importância das Variáveis

A análise dos coeficientes revela que:

- **Glicose** é a variável mais importante (coeficiente = 1.076)
- **BMI** tem alta importância (coeficiente = 0.706)
- **Pressão Arterial** tem efeito negativo (coeficiente = -0.101)
- Variáveis como **Insulina** e **Espessura da Pele** têm menor impacto

Conclusão

A análise demonstrou que é possível prever diabetes com uma performance razoável utilizando características clínicas básicas. O modelo de Regressão Logística foi selecionado como o melhor, com F1-score de 0.624 e ROC-AUC de 0.805. A variável mais importante para a predição é a concentração de glicose, seguida pelo índice de massa corporal.

O código implementado utiliza boas práticas de machine learning, incluindo validação cruzada, balanceamento de classes e pré-processamento adequado dos dados. Os resultados sugerem que o modelo pode ser útil como ferramenta de triagem inicial para identificação de pacientes com risco de diabetes.

Código Completo

```
import os
import json
import joblib
```

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, confusion_matrix, classification_report
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

xlsx_path = "Diabetes_Dataset_Missing_Data.xlsx"
out_dir = Path("output")
out_dir.mkdir(parents=True, exist_ok=True)

df = pd.read_excel(xlsx_path)
df.columns = [c.strip().lower() for c in df.columns]

expected = ['pregnancies', 'glucose', 'bloodpressure', 'skinthickness', 'insulin', 'bmi', 'familyhistorydiabetes', 'age', 'outcome']
missing_cols = [c for c in expected if c not in df.columns]
if missing_cols:
    raise ValueError(f"Colunas ausentes no XLSX: {missing_cols}")

df[expected] = df[expected].apply(pd.to_numeric, errors='coerce')

zeros_as_nan = ['glucose', 'bloodpressure', 'skinthickness', 'insulin', 'bmi']
for c in zeros_as_nan:
    df.loc[df[c] == 0, c] = np.nan

num_cols = ['pregnancies', 'glucose', 'bloodpressure', 'skinthickness', 'insulin', 'bmi', 'familyhistorydiabetes', 'age']
target_col = 'outcome'
X = df[num_cols].copy()
y = df[target_col].copy().astype(int)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42, stratify=y)

preprocess = ColumnTransformer([
    ("num", Pipeline([("imputer", SimpleImputer(strategy="median")), ("scaler", StandardScaler())]), num_cols)
])

models = {
    "logreg": {
        "pipeline": Pipeline([("preprocess", preprocess), ("clf", LogisticRegression(max_iter=1000, class_weight="balanced", solver="lbfgs"))]),
        "param_grid": {"clf__C": [0.1, 1, 10]}
    },
    "rf": {
        "pipeline": Pipeline([("preprocess", preprocess), ("clf", RandomForestClassifier(class_weight="balanced", random_state=42))]),
        "param_grid": {"clf__n_estimators": [100, 200, 300], "clf__max_depth": [None, 10, 20]}
    }
}

rows = []
best = {"name": None, "estimator": None, "f1": -1.0, "best_params": None}

for name, spec in models.items():
    gs = GridSearchCV(spec["pipeline"], spec["param_grid"], scoring="f1", cv=5, n_jobs=-1)
    gs.fit(X_train, y_train)
    y_pred = gs.predict(X_test)
    y_proba = gs.predict_proba(X_test)[:,1] if hasattr(gs.best_estimator_.named_steps["clf"], "predict_proba") else None
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, zero_division=0)
    rec = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)
    auc = roc_auc_score(y_test, y_proba) if y_proba is not None else np.nan
    rows.append({
        "name": name,
        "estimator": gs.best_estimator_.named_steps["clf"].__class__.__name__,
        "f1": f1,
        "best_params": gs.best_params_,
        "acc": acc,
        "prec": prec,
        "rec": rec,
        "auc": auc
    })

rows.sort(key=lambda x: x["f1"], reverse=True)
best = rows[0]

```

```

acc = roc_auc_score(y_test, y_pred) if y_pred is not None else np.NaN
rows.append({"model": name, "best_params": json.dumps(gs.best_params_), "accuracy": acc, "precision": prec, "recall": rec, "f1":
if f1 > best["f1"]:
    best["name"] = name
    best["estimator"] = gs.best_estimator_
    best["f1"] = f1
    best["best_params"] = gs.best_params_

summary = pd.DataFrame(rows).sort_values("f1", ascending=False)
summary.to_csv(out_dir / "models_summary.csv", index=False)

joblib.dump(best["estimator"], out_dir / "best_model.joblib")

y_pred = best["estimator"].predict(X_test)
y_proba = best["estimator"].predict_proba(X_test)[:,:1] if hasattr(best["estimator"].named_steps["clf"], "predict_proba") else None
report_txt = classification_report(y_test, y_pred, zero_division=0)
with open(out_dir / "classification_report.txt", "w") as f:
    f.write(report_txt)

cm = confusion_matrix(y_test, y_pred)
fig = plt.figure()
plt.imshow(cm, aspect="auto")
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("True")
for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        plt.text(j, i, cm[i, j], ha="center", va="center")
plt.tight_layout()
plt.savefig(out_dir / "confusion_matrix.png")
plt.close(fig)

if y_proba is not None:
    fpr, tpr, thr = roc_curve(y_test, y_proba)
    ap = average_precision_score(y_test, y_proba)
    pr_x, pr_y, _ = precision_recall_curve(y_test, y_proba)
    fig = plt.figure()
    plt.plot(fpr, tpr)
    plt.plot([0,1],[0,1],"--")
    plt.title("ROC Curve")
    plt.xlabel("FPR")
    plt.ylabel("TPR")
    plt.tight_layout()
    plt.savefig(out_dir / "roc_curve.png")
    plt.close(fig)
    fig = plt.figure()
    plt.plot(pr_x, pr_y)
    plt.title(f"Precision-Recall Curve (AP={ap:.3f})")
    plt.xlabel("Recall")
    plt.ylabel("Precision")
    plt.tight_layout()
    plt.savefig(out_dir / "pr_curve.png")
    plt.close(fig)

if best["name"] == "rf":
    rf = best["estimator"].named_steps["clf"]
    imp = pd.DataFrame({"feature": num_cols, "importance": rf.feature_importances_}).sort_values("importance", ascending=False)
    imp.to_csv(out_dir / "feature_importances_randomforest.csv", index=False)
else:
    logreg = best["estimator"].named_steps["clf"]
    pre = best["estimator"].named_steps["preprocess"]
    f_names = pre.get_feature_names_out().tolist()
    coef = pd.DataFrame({"feature": f_names, "coefficient": logreg.coef_.ravel()}).sort_values("coefficient", key=lambda s: s.abs(),
    coef.to_csv(out_dir / "coefficients_logreg.csv", index=False)

corr = df[num_cols + [target_col]].corr(numeric_only=True)

```

```
fig = plt.figure(figsize=(6,4))
plt.imshow(corr.values, aspect="auto")
plt.xticks(range(len(corr.columns)), corr.columns, rotation=90)
plt.yticks(range(len(corr.index)), corr.index)
plt.title("Correlation Heatmap")
plt.colorbar()
plt.tight_layout()
plt.savefig(out_dir / "correlation_heatmap.png")
plt.close(fig)
```

Arquivos Gerados

O código gera os seguintes arquivos na pasta `output/` :

- `models_summary.csv` - Resumo dos modelos testados
- `best_model.joblib` - Melhor modelo salvo
- `classification_report.txt` - Relatório de classificação
- `confusion_matrix.png` - Matriz de confusão
- `roc_curve.png` - Curva ROC
- `pr_curve.png` - Curva Precision-Recall
- `correlation_heatmap.png` - Mapa de calor de correlações
- `coefficients_logreg.csv` - Coeficientes da regressão logística